

Speech Recognition

Exercises NLTK2 on Stemming, Lemmatizing and PoS Tagging

Prof. Dr.-Ing. Udo Garmann

Part 01

Create a wordlist with at least the following words:

```
1 [ 'cats', 'fish', 'runs', 'played', 'gotten', 'feed', 'shed', 'emptied', 'proceed',  
   'generously', 'strictly', 'modernization']
```

Use the NLTK's Porter stemmer and one other stemmer to stem every word in the list. Are there differences in the results?

Find 3 examples where one stemmer returns a better result than the other.

Solution

```
1 # see https://www.nltk.org/howto/stem.html  
2 from nltk.stem.porter import *  
3 stemmer1 = PorterStemmer()  
4 stemmer2 = LancasterStemmer()  
5 wl = [ 'cats', 'fish', 'runs', 'played', 'gotten', 'feed', 'shed', 'emptied', 'proceed',  
        'generously', 'strictly', 'modernization']  
6 stems1 = [stemmer1.stem(word) for word in wl]  
7 stems2 = [stemmer2.stem(word) for word in wl]  
8 print("Stems from stemmer 1 - Porter:")  
9 print(stems1)  
10 # ['cat', 'fish', 'run', 'play', 'gotten', 'feed', 'shed', 'empti', 'proceed', 'gener',  
    'strictli', 'modern']  
11 print("Stems from stemmer 2 - Lancaster:")  
12 print(stems2)  
13 # ['cat', 'fish', 'run', 'play', 'got', 'fee', 'shed', 'empty', 'process', 'gen',  
    'strictly', 'modern']
```

Part 02

Take words from the wordlist of the previous exercise and lemmatize the words. Compare the results with the results of stemming.

Find 3 examples where lemmatizer returns a better result than the Porter Stemmer.

Solution

```
1 from nltk.stem import WordNetLemmatizer  
2 lemmatizer = WordNetLemmatizer()  
3 wl = [ 'cats', 'fish', 'runs', 'played', 'gotten', 'feed', 'shed', 'emptied', 'proceed',  
        'generously', 'strictly', 'modernization']  
4 lems = [lemmatizer.lemmatize(word) for word in wl]
```

```

5 print(lems)
6 # ['cat', 'fish', 'run', 'played', 'gotten', 'feed', 'shed', 'emptied', 'proceed',
   'generously', 'strictly', 'modernization']
7 for w in wl:
8     print(lemmatizer.lemmatize(w))
9     print(lemmatizer.lemmatize(w, 'n'))
10    print(lemmatizer.lemmatize(w, 'v'))
11    print(lemmatizer.lemmatize(w, 'a'))

```

Part 03

Write programs to process the Brown Corpus and find answers to the following questions:

- 1) Which nouns are more common in their plural form, rather than their singular form? (Only consider regular plurals, formed with the -s suffix.)
- 2) Which word has the greatest number of distinct tags. What are they, and what do they represent?
- 3) List tags in order of decreasing frequency. What do the 20 most frequent tags represent?
- 4) Which tags are nouns most commonly found after? What do these tags represent?

Part 04

Generate some statistics for tagged data to answer the following questions:

- 1) What proportion of word types are always assigned the same part-of-speech tag?
- 2) How many words are ambiguous, in the sense that they appear with at least two tags?
- 3) What percentage of word tokens in the Brown Corpus involve these ambiguous words?

Practical

Part 01

Your chatbot should analyse the input. Apply a tagger to an input sentence and add the tags to the output in the chatbot. Start with the `pos_tag()` function. You may test another one later. Compare the results with the results of the grammar parsing from the former exercise.